МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

до лабораторної роботи № 1
з дисципліни «Машинне навчання»
Тема: «Лінійна регресія, оптимізація та статистичний аналіз»

Виконала: студентка групи ФЕІ-34
Ксьондз У.О.
Перевірив: доц. Сінькевич О.О.

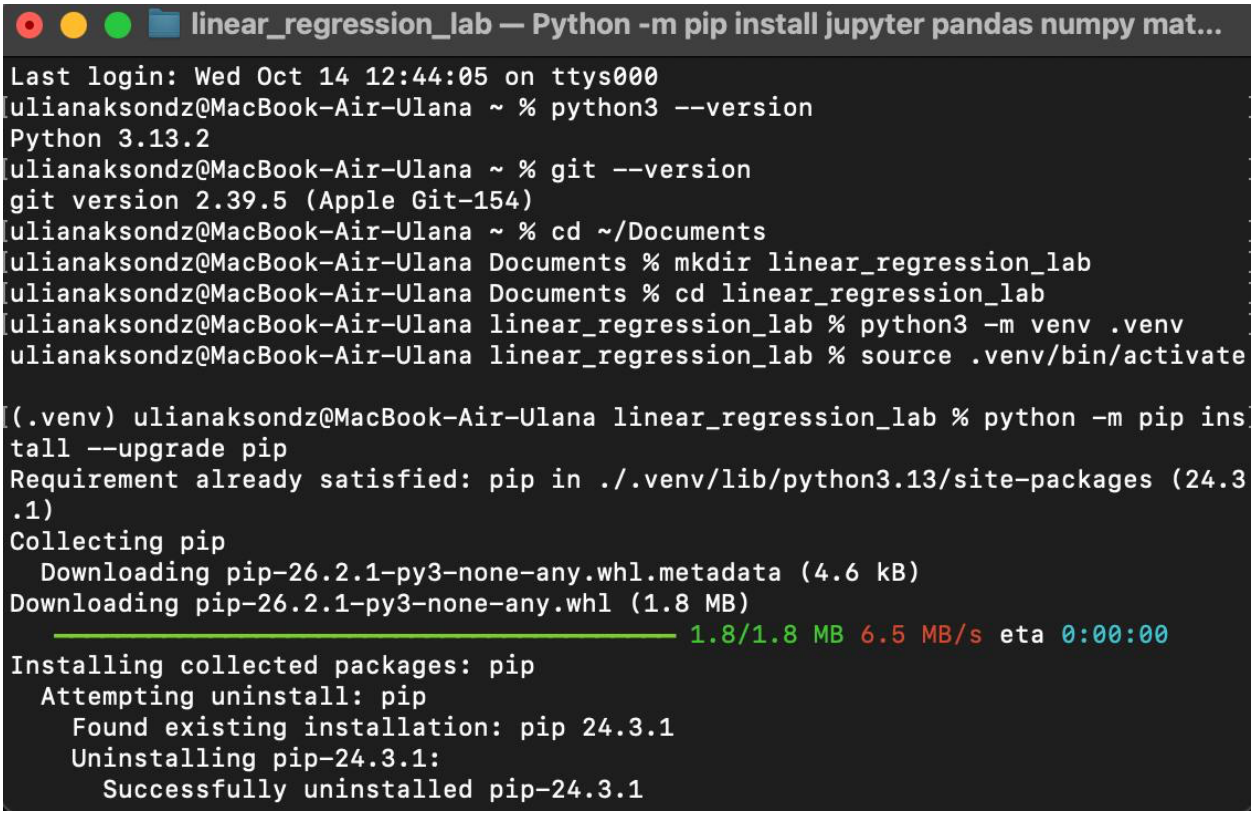
Львів - 2026

Мета завдання

Зрозуміти архітектуру лінійної регресії, навчитися самостійно реалізовувати алгоритми оптимізації, обирати інформативні ознаки, обчислювати метрики якості та перевіряти статистичні припущення моделі.

Хід роботи:

Створюємо папку лабораторної та встановлюємо бібліотеки



Імпортую бібліотеки

Лабораторна робота: Лінійна регресія

Дисципліна: Машинне навчання

Мета роботи: дослідити принципи роботи лінійної регресії, реалізувати алгоритми градієнтного спуску, оцінити якість моделі за допомогою метрик та перевірити статистичні припущення.

[4]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

from IPython.display import display

print("Бібліотеки успішно імпортовано!")
print("Версія NumPy:", np.__version__)
print("Версія Pandas:", pd.__version__)
print("Версія PyTorch:", torch.__version__)
```

```
Бібліотеки успішно імпортовано!
Версія NumPy: 2.5.4
Версія Pandas: 3.0.6
Версія PyTorch: 2.14.1
```

Датасет завантажився. Маємо 1460 записів і 81 стовпець

jupyter linear_regression_lab Last Checkpoint: 28 seconds ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
OpenML доступний, HTTP: 200
```

[14]:
```
import ssl
import certifi
import urllib.request
from sklearn.datasets import fetch_openml

ssl_context = ssl.create_default_context(cafile=certifi.where())

opener = urllib.request.build_opener(
    urllib.request.HTTPSHandler(context=ssl_context)
)

urllib.request.install_opener(opener)

housing = fetch_openml(
    name="house_prices",
    as_frame=True,
    parser="auto"
)

df = housing.frame.copy()

print("Розмір датасету:", df.shape)
display(df.head())
```

```
Розмір датасету: (1460, 81)
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoS
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	

5 rows × 81 columns

Первинний аналіз даних

На цьому етапі досліджується набір даних Ames Housing, який містить характеристики житлових будинків та інформацію про ціну їх продажу.

Мета аналізу - визначити структуру набору, типи ознак, наявність пропущених значень і взаємозв'язки між характеристиками нерухомості та її вартістю.

[15]:
```
print("Розмір датасету:", df.shape)

print("\nТипи даних:")
display(df.dtypes.value_counts().to_frame("Кількість стовпців"))

print("\nІнформація про стовпці:")
df.info()
```

```
Розмір датасету: (1460, 81)

Типи даних:
```

	Кількість стовпців
str	43
int64	35
float64	3

```
Інформація про стовпці:
<class 'pandas.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 81 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Id              1460 non-null   int64
 1   MSSubClass      1460 non-null   int64
 2   MSZoning        1460 non-null   str
 3   LotFrontage     1201 non-null   float64
 4   LotArea         1460 non-null   int64
 5   Street          1460 non-null   str
 6   Alley           91 non-null     str
 7   LotShape        1460 non-null   str
 8   LandContour     1460 non-null   str
 9   Utilities       1460 non-null   str
 10  LotConfig       1460 non-null   str
 11  LandSlope       1460 non-null   str
 12  Neighborhood    1460 non-null   str
 13  Condition1      1460 non-null   str
 14  Condition2      1460 non-null   str
```

Таблиця: таблиця покаже середні значення, стандартні відхилення, мінімальні й максимальні значення та квартилі числових ознак.

jupyter linear_regression_lab Last Checkpoint: 2 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
[16]: display(df.describe().T)
```

	count	mean	std	min	25%	50%	75%	max
Id	1460.0	730.500000	421.610009	1.0	365.75	730.5	1095.25	1460.0
MSSubClass	1460.0	56.897260	42.300571	20.0	20.00	50.0	70.00	190.0
LotFrontage	1201.0	70.049958	24.284752	21.0	59.00	69.0	80.00	313.0
LotArea	1460.0	10516.828082	9981.264932	1300.0	7553.50	9478.5	11601.50	215245.0
OverallQual	1460.0	6.099315	1.382997	1.0	5.00	6.0	7.00	10.0
OverallCond	1460.0	5.575342	1.112799	1.0	5.00	5.0	6.00	9.0
YearBuilt	1460.0	1971.267808	30.202904	1872.0	1954.00	1973.0	2000.00	2010.0
YearRemodAdd	1460.0	1984.865753	20.645407	1950.0	1967.00	1994.0	2004.00	2010.0
MasVnrArea	1452.0	103.685262	181.066207	0.0	0.00	0.0	166.00	1600.0
BsmtFinSF1	1460.0	443.639726	456.098091	0.0	0.00	383.5	712.25	5644.0
BsmtFinSF2	1460.0	46.549315	161.319273	0.0	0.00	0.0	0.00	1474.0
BsmtUnfSF	1460.0	567.240411	441.866955	0.0	223.00	477.5	808.00	2336.0
TotalBsmtSF	1460.0	1057.429452	438.705324	0.0	795.75	991.5	1298.25	6110.0
1stFlrSF	1460.0	1162.626712	386.587738	334.0	882.00	1087.0	1391.25	4692.0
2ndFlrSF	1460.0	346.992466	436.528436	0.0	0.00	0.0	728.00	2065.0
LowQualFinSF	1460.0	5.844521	48.623081	0.0	0.00	0.0	0.00	572.0
GrLivArea	1460.0	1515.463699	525.480383	334.0	1129.50	1464.0	1776.75	5642.0
BsmtFullBath	1460.0	0.425342	0.518911	0.0	0.00	0.0	1.00	3.0
BsmtHalfBath	1460.0	0.057534	0.238753	0.0	0.00	0.0	0.00	2.0
FullBath	1460.0	1.565068	0.550916	0.0	1.00	2.0	2.00	3.0
HalfBath	1460.0	0.382877	0.502885	0.0	0.00	0.0	1.00	2.0
BedroomAbvGr	1460.0	2.866438	0.815778	0.0	2.00	3.0	3.00	8.0

У датасеті є ознаки з великою кількістю пропусків

jupyter linear_regression_lab Last Checkpoint: 2 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
[17]: missing = pd.DataFrame({
          "Пропущено": df.isna().sum(),
          "Відсоток (%)": df.isna().mean() * 100
      })

      missing = missing[missing["Пропущено"] > 0]
      missing = missing.sort_values("Відсоток (%)", ascending=False)

      print("Кількість стовпців із пропусками:", len(missing))
      display(missing.head(20))
```

```
Кількість стовпців із пропусками: 19
```

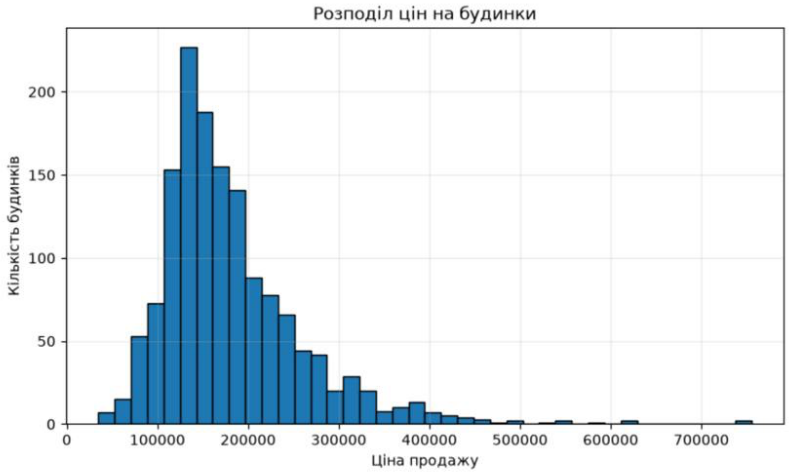
	Пропущено	Відсоток (%)
PoolQC	1453	99.520548
MiscFeature	1406	96.301370
Alley	1369	93.767123
Fence	1179	80.753425
FireplaceQu	690	47.260274
LotFrontage	259	17.739726
GarageType	81	5.547945
GarageYrBlt	81	5.547945
GarageFinish	81	5.547945
GarageQual	81	5.547945
GarageCond	81	5.547945
BsmtExposure	38	2.602740
BsmtFinType2	38	2.602740
BsmtFinType1	37	2.534247
BsmtCond	37	2.534247
BsmtQual	37	2.534247

На графіку бачимо, у якому діапазоні найчастіше продаються будинки та чи є дорогі об'єкти, що відрізняються від основної маси.

```
[18]: print(df["SalePrice"].describe())

plt.figure(figsize=(9, 5))
plt.hist(df["SalePrice"], bins=40, edgecolor="black")
plt.title("Розподіл цін на будинки")
plt.xlabel("Ціна продажу")
plt.ylabel("Кількість будинків")
plt.grid(alpha=0.25)
plt.show()
```

```
count      1460.000000
mean     180921.195890
std       79442.502883
min       34900.000000
25%      129975.000000
50%      163000.000000
75%      214000.000000
max      755000.000000
Name: SalePrice, dtype: float64
```




```
[ ]:
```

Кореляція числових ознак із ціною

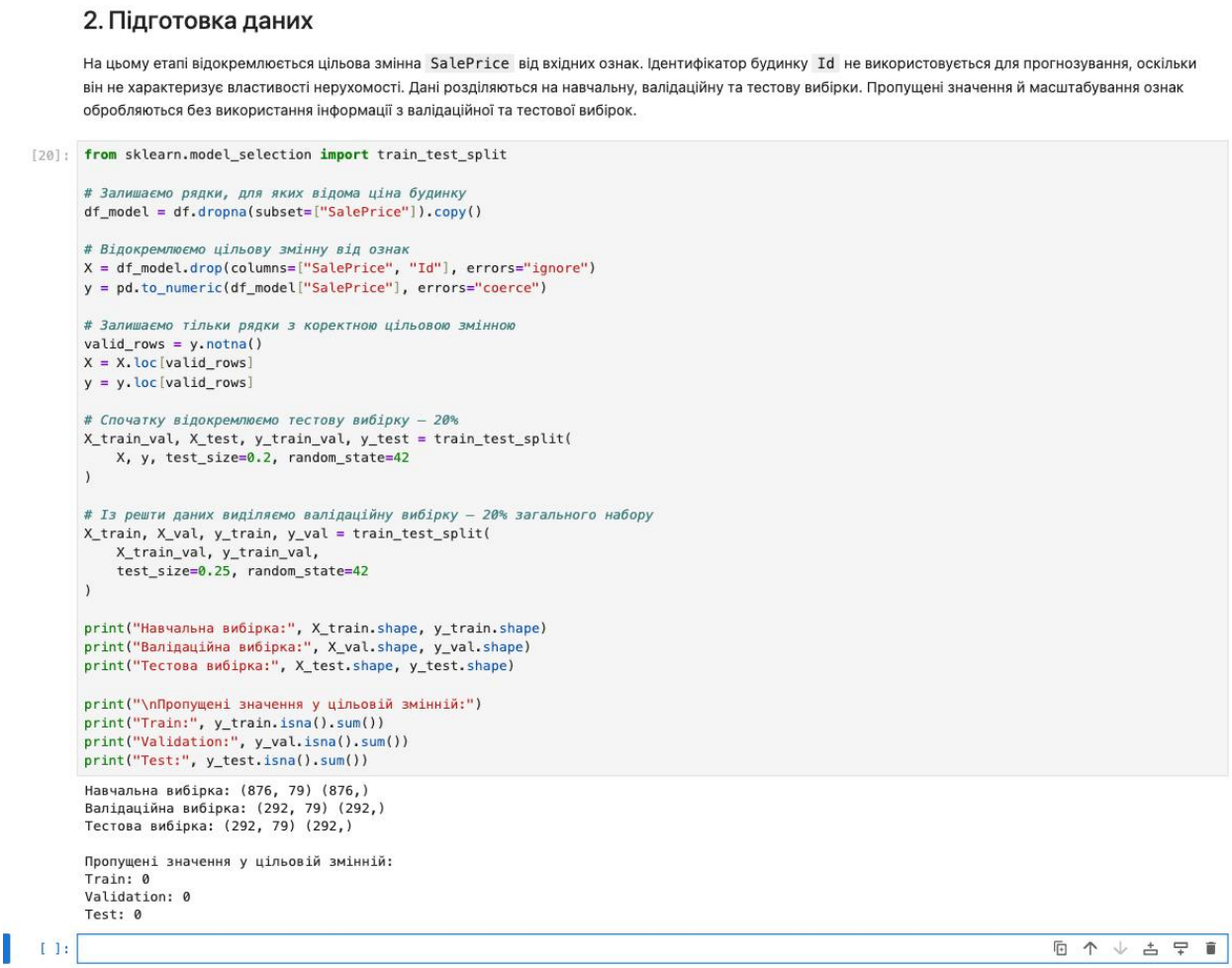
Кореляція близька до 1 означає сильний позитивний лінійний зв'язок, близька до -1 -- негативний, а близька до 0 -- слабкий лінійний зв'язок.

Результати правильні. Найсильніший лінійний зв’язок із ціною має ознака OverallQual - загальна оцінка якості будинку. Далі йдуть GrLivArea - житлова площа над рівнем землі та GarageCars - місткість гаража.

Це позитивна кореляція: зі збільшенням цих показників ціна будинку зазвичай зростає. Однак кореляція не означає причинно-наслідкового зв’язку.

Підготовка даних

На цьому етапі відокремлюється цільова змінна SalePrice від вхідних ознак. Ідентифікатор будинку Id не використовується для прогнозування, оскільки він не характеризує властивості нерухомості. Дані розділяються на навчальну, валідаційну та тестову вибірки. Пропущені значення й масштабування ознак обробляються без використання інформації з валідаційної та тестової вибірок.



2. Підготовка даних

На цьому етапі відокремлюється цільова змінна `SalePrice` від вхідних ознак. Ідентифікатор будинку `Id` не використовується для прогнозування, оскільки він не характеризує властивості нерухомості. Дані розділяються на навчальну, валідаційну та тестову вибірки. Пропущені значення й масштабування ознак обробляються без використання інформації з валідаційної та тестової вибірок.

[20]:
```
from sklearn.model_selection import train_test_split

# Залишаємо рядки, для яких відома ціна будинку
df_model = df.dropna(subset=["SalePrice"]).copy()

# Відокремлюємо цільову змінну від ознак
X = df_model.drop(columns=["SalePrice", "Id"], errors="ignore")
y = pd.to_numeric(df_model["SalePrice"], errors="coerce")

# Залишаємо тільки рядки з коректною цільовою змінною
valid_rows = y.notna()
X = X.loc[valid_rows]
y = y.loc[valid_rows]

# Спочатку відокремлюємо тестову вибірку — 20%
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Із решти даних виділяємо валідаційну вибірку — 20% загального набору
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val,
    test_size=0.25, random_state=42
)

print("Навчальна вибірка:", X_train.shape, y_train.shape)
print("Валідаційна вибірка:", X_val.shape, y_val.shape)
print("Тестова вибірка:", X_test.shape, y_test.shape)

print("\nПропущені значення у цільовій змінній:")
print("Train:", y_train.isna().sum())
print("Validation:", y_val.isna().sum())
print("Test:", y_test.isna().sum())
```

```
Навчальна вибірка: (876, 79) (876,)
Валідаційна вибірка: (292, 79) (292,)
Тестова вибірка: (292, 79) (292,)

Пропущені значення у цільовій змінній:
Train: 0
Validation: 0
Test: 0
```

[ ]:

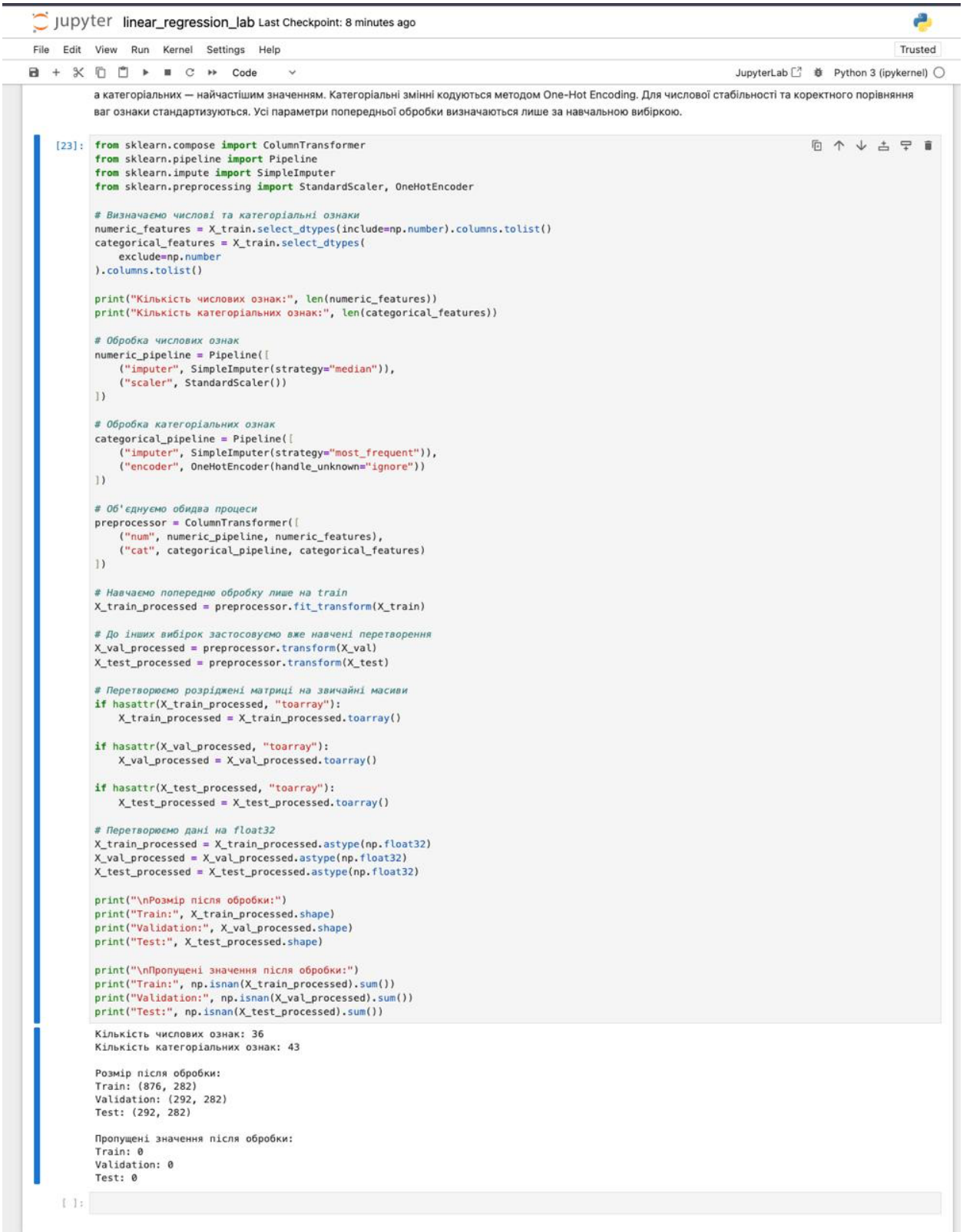
Маємо:

- 36 числових і 43 категоріальні початкові ознаки.

- 282 ознаки після One-Hot Encoding і стандартизації.

- 876 навчальних, 292 валідаційних і 292 тестових прикладів.

- Пропущених значень після обробки - 0 у всіх вибірках:



Реалізація лінійної регресії з нуля

4. Реалізація лінійної регресії з нуля

Лінійна регресія використовується для прогнозування числової цільової змінної на основі вхідних ознак. У цій роботі модель реалізується самостійно за допомогою PyTorch без використання готових регресійних моделей.

Навчання виконується шляхом мінімізації середньоквадратичної помилки (MSE). Для оптимізації параметрів використовуються пакетний градієнтний спуск (Batch Gradient Descent), стохастичний градієнтний спуск (SGD) і мініпакетний градієнтний спуск (Mini-batch Gradient Descent).

[24]:
```
import torch

# Фіксуємо випадковість для відтворюваності результатів
np.random.seed(42)
torch.manual_seed(42)

# Перетворюємо цільову змінну на числовий формат
y_train_np = y_train.to_numpy(dtype=np.float32).reshape(-1, 1)
y_val_np = y_val.to_numpy(dtype=np.float32).reshape(-1, 1)
y_test_np = y_test.to_numpy(dtype=np.float32).reshape(-1, 1)

# Масштабуємо цільову змінну для стабільнішого навчання.
# Параметри обчислюємо лише на навчальній вибірці.
y_mean = y_train_np.mean()
y_std = y_train_np.std()

y_train_scaled = (y_train_np - y_mean) / y_std
y_val_scaled = (y_val_np - y_mean) / y_std
y_test_scaled = (y_test_np - y_mean) / y_std

# Перетворюємо дані на тензори PyTorch
X_train_tensor = torch.tensor(X_train_processed, dtype=torch.float32)
X_val_tensor = torch.tensor(X_val_processed, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test_processed, dtype=torch.float32)

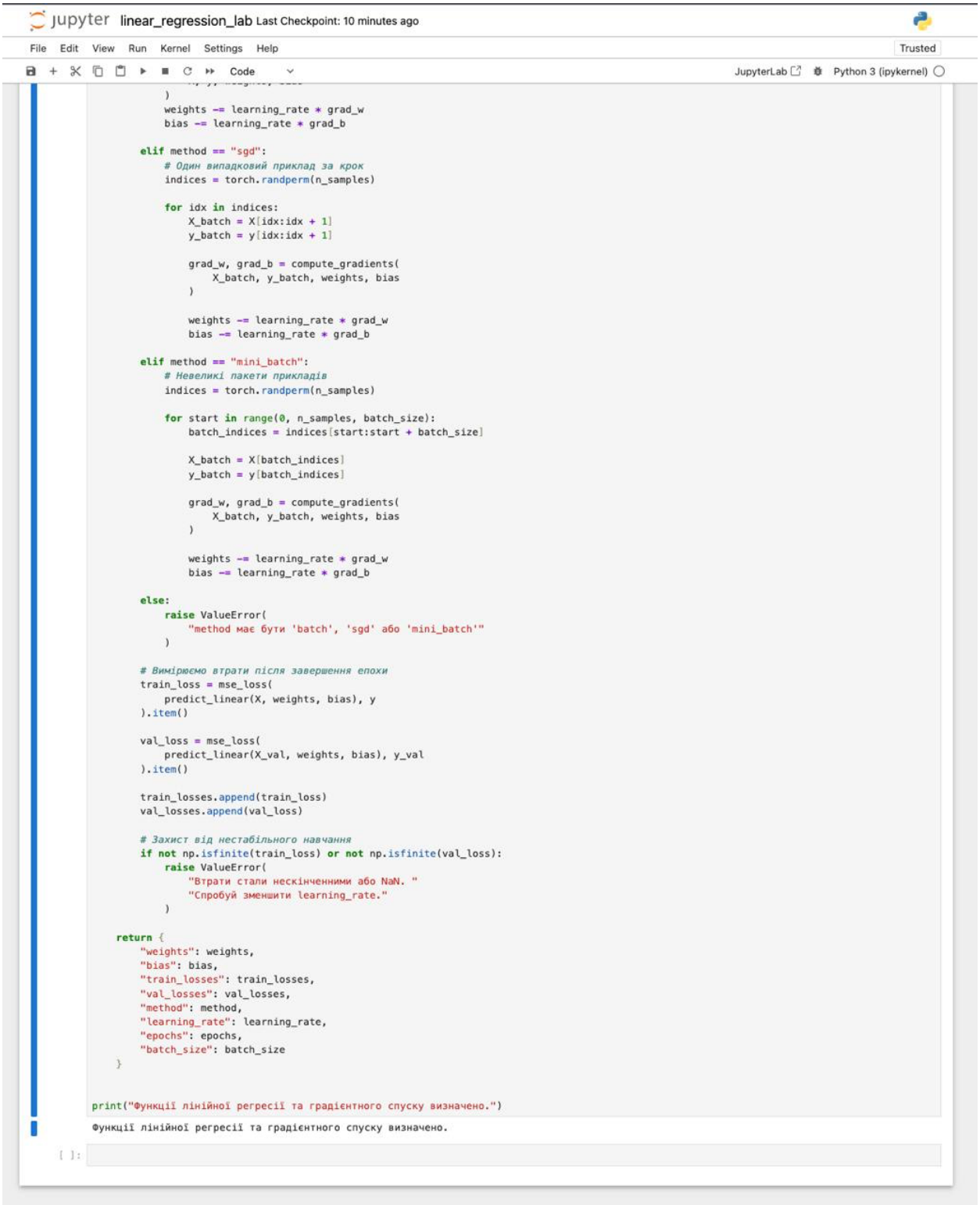
y_train_tensor = torch.tensor(y_train_scaled, dtype=torch.float32)
y_val_tensor = torch.tensor(y_val_scaled, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test_scaled, dtype=torch.float32)

print("PyTorch:", torch.__version__)
print("Train X:", X_train_tensor.shape)
print("Train y:", y_train_tensor.shape)
print("Validation X:", X_val_tensor.shape)
print("Test X:", X_test_tensor.shape)
print("Середня ціна у train:", round(float(y_mean), 2))
print("Стандартне відхилення ціни:", round(float(y_std), 2))
```

```
PyTorch: 2.14.1
Train X: torch.Size([876, 282])
Train y: torch.Size([876, 1])
Validation X: torch.Size([292, 282])
Test X: torch.Size([292, 282])
Середня ціна у train: 181233.97
Стандартне відхилення ціни: 77177.85
```

[ ]:

Власна реалізація алгоритму



Реалізовано:

- Прогноз: зважена сума ознак і вільний член.

- MSE: середній квадрат різниці між прогнозом і справжньою ціною.

- Градієнти: обчислюються вручну за формулами похідних.

- Три методи: batch, sgd, mini_batch.

- Контроль якості: після кожної епохи вимірюються втрати на навчальній і валідаційній вибірках.

Для SGD оновлення відбувається після кожного прикладу, а для mini-batch - після кожної групи з 32 прикладів.

Навчання моделі

[26]:
```
# Параметри першого експерименту
EPOCHS = 100
LEARNING_RATE = 0.01
BATCH_SIZE = 32

results = {}

methods = {
    "Batch GD": "batch",
    "SGD": "sgd",
    "Mini-batch GD": "mini_batch"
}

for name, method in methods.items():
    print(f"\nНавчання: {name}")

    result = train_linear_regression(
        X_train_tensor,
        y_train_tensor,
        X_val_tensor,
        y_val_tensor,
        method=method,
        learning_rate=LEARNING_RATE,
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        seed=42
    )

    results[name] = result

    print(f"Train MSE: {result['train_losses'][-1]:.6f}")
    print(f"Validation MSE: {result['val_losses'][-1]:.6f}")

print("\nНавчання всіх трьох моделей завершено.")
```

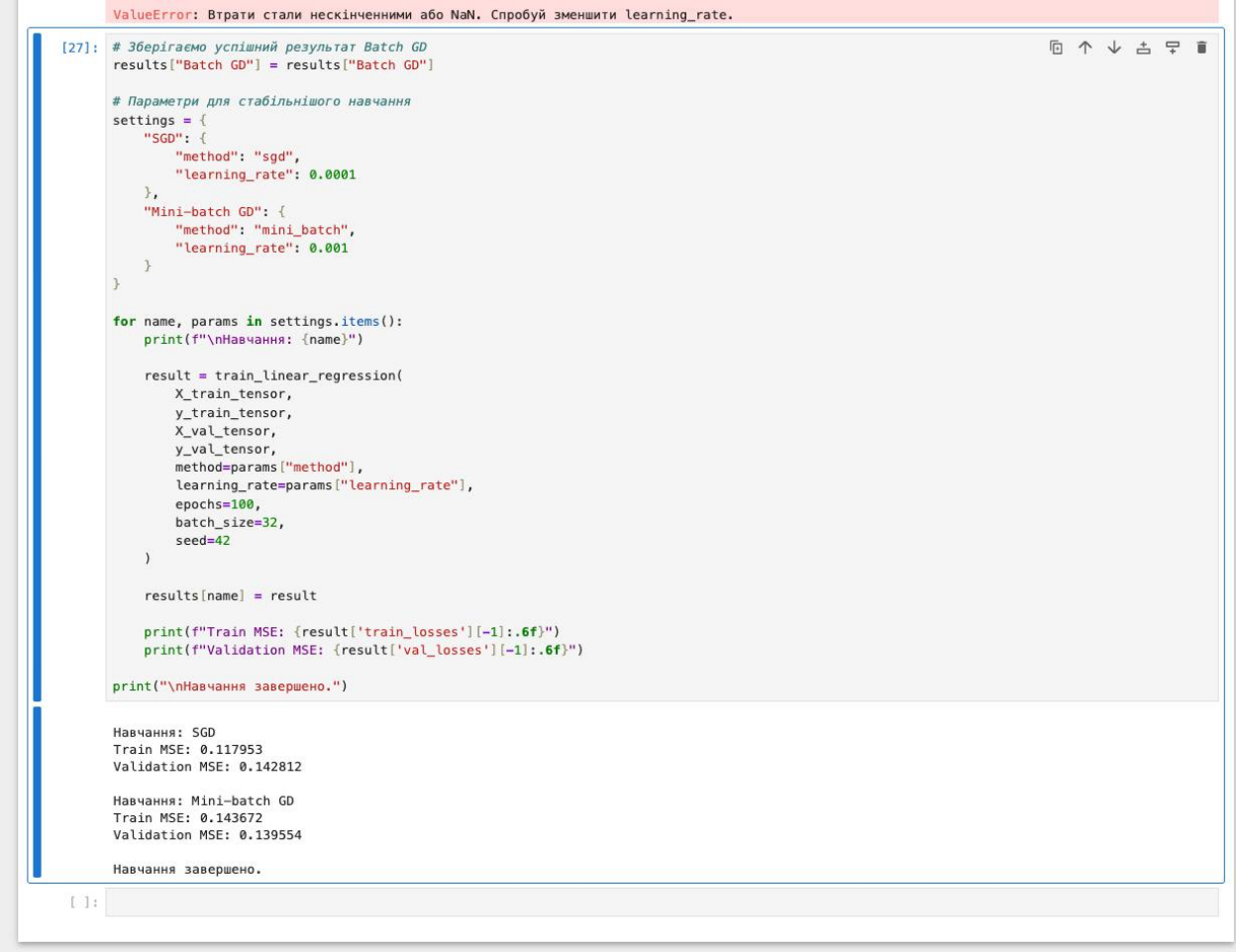
```
Навчання: Batch GD
Train MSE: 0.171727
Validation MSE: 0.149698

Навчання: SGD
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[26], line 17
     13
     14 for name, method in methods.items():
     15     print(f"\nНавчання: {name}")
     16
---> 17     result = train_linear_regression(
     18         X_train_tensor,
     19         y_train_tensor,
     20         X_val_tensor,

Cell In[25], line 108, in train_linear_regression(X, y, X_val, y_val, method, learning_rate, epochs, batch_size, seed)
    104         val_losses.append(val_loss)
    105
    106         # Захист від нестабільного навчання
    107         if not np.isfinite(train_loss) or not np.isfinite(val_loss):
--> 108             raise ValueError(
    109                 "Втрати стали нескінченними або NaN. "
    110                 "Спробуй зменшити learning_rate."
    111             )

ValueError: Втрати стали нескінченними або NaN. Спробуй зменшити learning_rate.
```

[27]:
```
# Зберігаємо успішний результат Batch GD
results["Batch GD"] = results["Batch GD"]
```



```
ValueError: Втрати стали нескінченними або NaN. Спробуй зменшити learning_rate.
```

```
[27]: # Зберігаємо успішний результат Batch GD
results["Batch GD"] = results["Batch GD"]

# Параметри для стабільнішого навчання
settings = {
    "SGD": {
        "method": "sgd",
        "learning_rate": 0.0001
    },
    "Mini-batch GD": {
        "method": "mini_batch",
        "learning_rate": 0.001
    }
}

for name, params in settings.items():
    print(f"\nНавчання: {name}")

    result = train_linear_regression(
        X_train_tensor,
        y_train_tensor,
        X_val_tensor,
        y_val_tensor,
        method=params["method"],
        learning_rate=params["learning_rate"],
        epochs=100,
        batch_size=32,
        seed=42
    )

    results[name] = result

    print(f"Train MSE: {result['train_losses'][-1]:.6f}")
    print(f"Validation MSE: {result['val_losses'][-1]:.6f}")

print("\nНавчання завершено.")
```

```
Навчання: SGD
Train MSE: 0.117953
Validation MSE: 0.142812

Навчання: Mini-batch GD
Train MSE: 0.143672
Validation MSE: 0.139554

Навчання завершено.
```

```
[ ]:
```

Усі три методи навчання відпрацювали успішно. Порівнюємо результати першого експерименту.

1. Порівняння результатів

Метод Train MSE Validation MSE

Batch GD 0.171727 0.149698

SGD 0.117953 0.142812

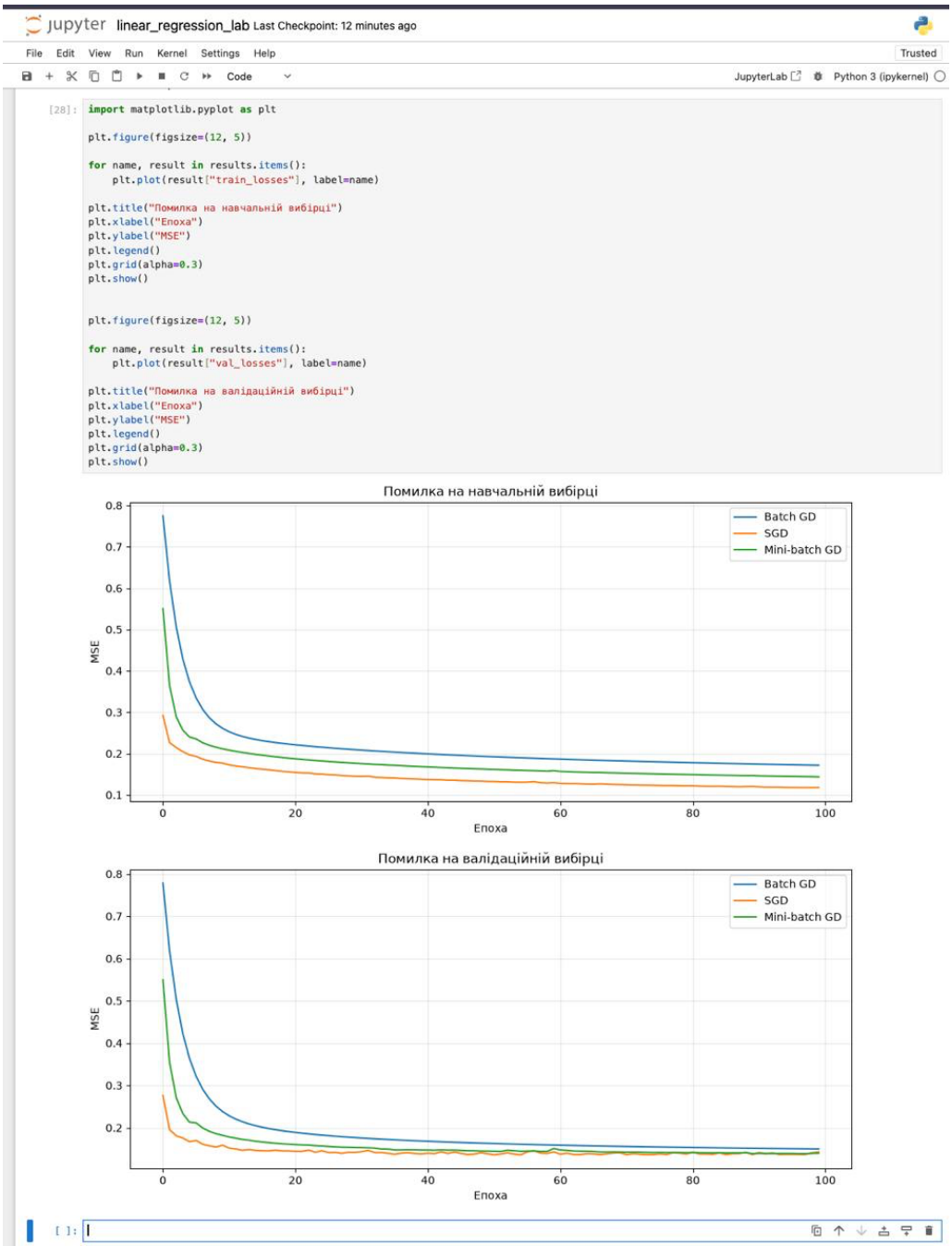
Mini-batch GD 0.143672 0.139554

MSE - середньоквадратична помилка. Чим менше її значення, тим точніші прогнози на відповідній вибірці.

- Batch GD: використовує всю навчальну вибірку для одного оновлення ваг за епоху.

- SGD: оновлює ваги після кожного окремого прикладу. Має найменшу помилку на тренувальних даних.

- Mini-batch GD: оновлює ваги невеликими групами прикладів. У нашому експерименті показав найменшу помилку на валідаційній вибірці, тому поки є найкращим за здатністю узагальнювати дані.



Оцінюємо якість моделей

```
[29]: from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
      import pandas as pd
      import numpy as np

      evaluation_results = []

      for name, result in results.items():
          weights = result["weights"]
          bias = result["bias"]

          for dataset_name, X_data, y_original in [
              ("Train", X_train_tensor, y_train),
              ("Validation", X_val_tensor, y_val),
              ("Test", X_test_tensor, y_test)
          ]:
              # Прогноз у стандартизованому масштабі
              with torch.no_grad():
                  predictions_scaled = predict_linear(
                      X_data, weights, bias
                  ).cpu().numpy().reshape(-1)

              # Повертаємо прогноз до цін у доларах
              predictions = predictions_scaled * y_std + y_mean
              actual = y_original.to_numpy(dtype=np.float64)

              evaluation_results.append({
                  "Метод": name,
                  "Вибірка": dataset_name,
                  "R²": r2_score(actual, predictions),
                  "MAE": mean_absolute_error(actual, predictions),
                  "RMSE": np.sqrt(mean_squared_error(actual, predictions))
              })

      evaluation_df = pd.DataFrame(evaluation_results)

      print(evaluation_df.round(3).to_string(index=False))
```

```
        Метод    Вибірка    R²       MAE      RMSE
     Batch GD      Train 0.828 18305.273 31982.481
     Batch GD Validation 0.851 19552.917 29860.736
     Batch GD       Test 0.834 20546.625 35713.926
          SGD      Train 0.882 16408.611 26506.169
          SGD Validation 0.858 18699.828 29165.889
          SGD       Test 0.848 20373.412 34132.738
Mini-batch GD      Train 0.856 17362.792 29253.577
Mini-batch GD Validation 0.861 18642.638 28831.275
Mini-batch GD       Test 0.845 20063.398 34470.635
```

```
[ ]:
```

1. Аналіз отриманих метрик

Метод Test R² Test MAE Test RMSE

Batch GD 0.834 $20 546.63 $35 713.93

SGD 0.848 $20 373.41 $34 132.74

Mini-batch GD 0.845 $20 063.40 $34 470.64

- SGD має найвище тестове (R^2 = 0.848) і найменше RMSE.

- Mini-batch GD має найменше MAE, тобто найменшу середню абсолютну помилку.

- Batch GD поступається двом іншим методам за всіма трьома тестовими метриками.

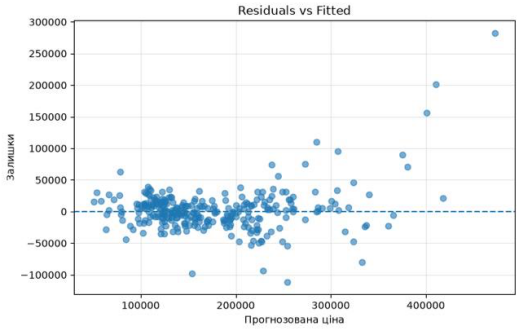
Значення (R^2 = 0.848) означає, що модель SGD пояснює приблизно 84,8% варіації цін у тестовій вибірці. Середня абсолютна помилка моделей становить близько 20 тисяч доларів.

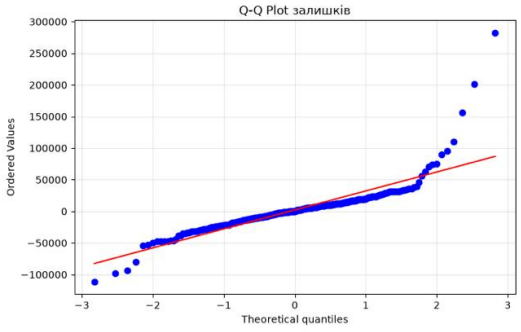
Різниця між тренувальними та тестовими показниками не виглядає надмірною, хоча сама по собі ще не гарантує відсутності перенавчання.

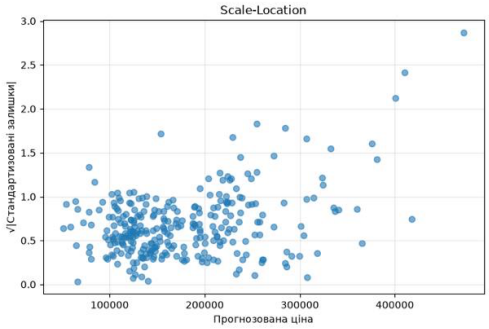
2. Перевіряємо залишки моделі

Залишок - це різниця між реальною ціною будинку та прогнозом моделі. Аналіз залишків допоможе перевірити, чи є ознаки нелінійності, нерівномірного розкиду помилок і відхилення від нормального розподілу.

Для цього візьмемо SGD, який має найвище тестове (R^2).

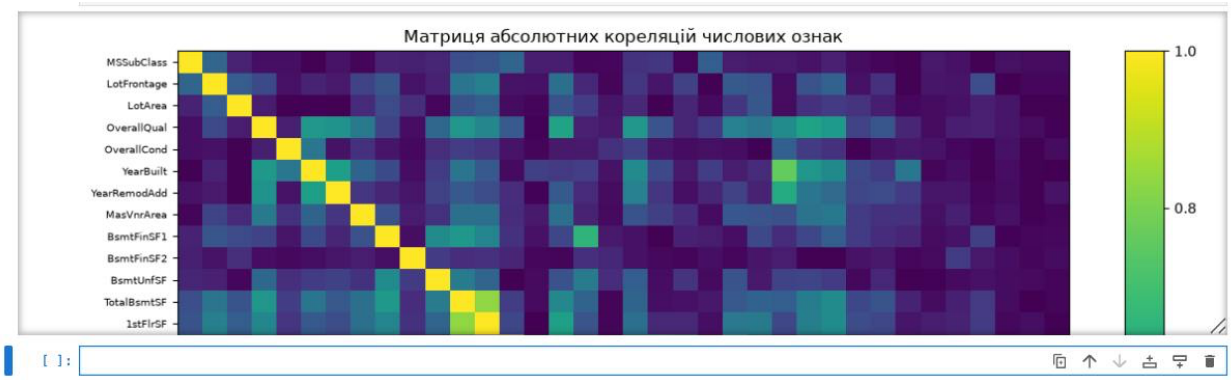






Статистика Shapiro-Wilk: 0.7742
p-value: 0.000000

Перевіряємо мультиколінеарність



У ході експерименту було реалізовано три методи навчання лінійної регресії: Batch Gradient Descent, Stochastic Gradient Descent і Mini-batch Gradient Descent. Найвище значення коефіцієнта детермінації на тестовій вибірці отримано для SGD - ($R^2 = 0,848$). Найменшу середню абсолютну похибку показав Mini-batch GD - приблизно 20063 долари. Результати свідчать про достатньо високу здатність моделей прогнозувати вартість будинків.

Тест Шапіро - Вілка виявив відхилення залишків від нормального розподілу $p < 0,05$. Це свідчить про порушення припущення щодо нормальності залишків, але саме по собі не означає низької точності прогнозування.

Виявлено 3 пари ознак із кореляцією понад 0.8. Це означає, що деякі характеристики будинків несуть схожу інформацію.

За VIF маємо:

- 12 ознак із VIF понад 5;

- 8 ознак із VIF понад 10;

- максимальний VIF -- аномально великий.

```
print("Функцію Ridge-регресії визначено.")
```

```
Функцію Ridge-регресії визначено.
```

[35]:
```
ridge_results = {}

for alpha in [0.001, 0.01, 0.1, 1.0]:
    model = train_ridge(
        X_train_tensor,
        y_train_tensor,
        X_val_tensor,
        y_val_tensor,
        learning_rate=0.001,
        epochs=100,
        alpha=alpha
    )

    ridge_results[alpha] = model

    print(
        f"alpha={alpha:<6} | "
        f"Train MSE={model['train_losses'][-1]:.6f} | "
        f"Validation MSE={model['val_losses'][-1]:.6f}"
    )

best_alpha = min(
    ridge_results,
    key=lambda a: ridge_results[a]["val_losses"][-1]
)

print(f"\nНайкраще alpha: {best_alpha}")
```

```
alpha=0.001  | Train MSE=0.267502 | Validation MSE=0.246245
alpha=0.01   | Train MSE=0.267672 | Validation MSE=0.246448
alpha=0.1    | Train MSE=0.269386 | Validation MSE=0.248495
alpha=1.0    | Train MSE=0.287681 | Validation MSE=0.269834

Найкраще alpha: 0.001
```

[36]:
```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np
import pandas as pd
import torch

# Найкраща Ridge-модель за Validation MSE
ridge_best = ridge_results[best_alpha]

with torch.no_grad():
    ridge_pred_scaled = (
        X_test_tensor @ ridge_best["weights"] + ridge_best["bias"]
    ).cpu().numpy().ravel()

# Повертаємо ціну до початкового масштабу
ridge_pred = ridge_pred_scaled * y_std + y_mean
y_actual = y_test.to_numpy(dtype=np.float64)

ridge_test_metrics = {
    "Модель": "Ridge",
    "Alpha": best_alpha,
    "R²": r2_score(y_actual, ridge_pred),
    "MAE": mean_absolute_error(y_actual, ridge_pred),
    "RMSE": np.sqrt(mean_squared_error(y_actual, ridge_pred))
}

print(pd.DataFrame([ridge_test_metrics]).round(3).to_string(index=False))
```

```
Модель  Alpha    R²       MAE      RMSE
 Ridge  0.001 0.737 24768.959 44954.925
```

[ ]:

Отже, Ridge показала гірший результат. Проте це не доводить, що регуляризація завжди погіршує прогнозування: її результат залежить від коефіцієнта регуляризації, швидкості навчання та кількості епох. У нашому експерименті Ridge навчалася з learning_rate=0.001 протягом 100 епох, тому вона могла не встигнути достатньо зблизитися з оптимумом.

GitHub: https://github.com/lianaksndz/linear-regression-lab.git

Висновки:

У ході лабораторної роботи було досліджено принципи побудови та навчання моделі лінійної регресії для прогнозування вартості житлової нерухомості на основі набору даних Ames Housing. Було виконано попередній аналіз даних, обробку пропущених значень, кодування категоріальних ознак, масштабування числових характеристик і розподіл даних на навчальну, валідаційну та тестову вибірки.

Було реалізовано три методи градієнтного спуску: Batch GD, SGD і Mini-batch GD. Найвище значення коефіцієнта детермінації на тестовій вибірці отримано для SGD - 0,848. Найменшу середню абсолютну помилку показав Mini-batch GD - приблизно 20 063 долари. Також було досліджено Ridge-регуляризацію, яка за обраних параметрів поступилася базовим моделям.

Аналіз кореляцій виявив три пари числових ознак із коефіцієнтом кореляції понад 0,8, що свідчить про наявність потенційної мультиколінеарності. Тест Шапіро - Вілка виявив статистично значуще відхилення залишків від нормального розподілу. Отже, побудована модель забезпечує достатньо високу точність прогнозування, проте результати аналізу залишків і взаємозалежності ознак вказують на можливості подальшого вдосконалення.